

Домашнее задание №1: Github, ветвление и git revert

Василин Владислав, гр. 604

Часть 1. Выполненные шаги и использованные команды

1) Создал репозиторий, README.md файл.

2) Настроил Git:

```
git config --global user.name "Vladislav Vasilin"
```

```
git config --global user.email vlad.vasilin.03@gmail.com
```

Результат работы git config --list:

```
....  
user.name=vladislav vasilin  
user.email=vlad.vasilin.03@gmail.com  
....
```

Клонировал репозиторий в локальную директорию на компьютере.

3) Создал ветку с помощью git switch -с HW1, создал три файла с текстом

touch hw1.txt → nano hw1.txt → ввожу желаемый текст ("This is an initial file.")

touch test_revert.txt → nano test_revert.txt → ввожу желаемый текст ("This is a file for testing revert.")

touch test_revert_merge.txt → nano test_revert_merge.txt → ввожу желаемый текст ("This is a file for testing revert and merge.")

4) Добавляем файлы в индекс и создаём первый коммит:

```
git add HW1/
```

```
git commit -m "Add initial files for HW1"
```

5) Пушим! 🌈🐼

```
git push -u origin HW1
```

6) Создал ветку testing, в ветке HW1 изменил файл hw1.txt, закоммитил и запустил изменения.

```
git branch testing
```

nano HW1/hw1.txt → ввожу желаемый текст ("This is an initial file. This is a change.")

```
git add HW1/hw1.txt
```

```
git commit -m "Update hw1 file"
```

```
git push
```

7) Ветка testing была создана до обновления hw1 файла, она указывает на предыдущий коммит "Add initial files for HW1".

8) Переключаемся на ветку testing, изменяем файл, коммит, пуш.

```
git switch testing
```

nano HW1/test_revert.txt → ввожу желаемый текст ("This is a file for testing revert. This is a change.")

```
git add HW1/test_revert.txt
```

```
git commit -m "Update file for revert testing"
```

```
git push -u origin testing
```

9) Перед пунктом 15 задания имеем вывод:

```
$ git log --oneline --graph --all --decorate
* 2384e2e (HEAD -> testing, origin/testing) update file for revert testing
| * 91ef8e6 (origin/HW1, HW1) update hw1 file
|/
* 1510ba6 Add initial files for HW1
* d274fe9 (origin/main, origin/HEAD, main) Initial commit
```

10) Слили ветки, запустили, результат git log --oneline --graph --all --decorate:

```
$ git log --oneline --graph --all --decorate
* 213529d (HEAD -> HW1, origin/HW1) Merge branch 'testing' into HW1
| \
| * 2384e2e (origin/testing, testing) update file for revert testing
| * | 91ef8e6 update hw1 file
|/
* 1510ba6 Add initial files for HW1
* d274fe9 (origin/main, origin/HEAD, main) Initial commit
```

То есть до слияния ветки HW1 и testing указывают на разные коммиты (разные файлы были изменены), после команды git merge testing был создан общий коммит, в результате чего ветка HW1 стала содержать изменения из обеих веток.

11) Отменяем merge-коммит. Из предыдущего пункта видим, что идентификатор этого коммита 213529d:

```
git revert -m 1 213529d
```

```
git push
```

12) Переключаемся на testing, редактируем третий файл, добавляем изменения в индекс, коммитим и пушим:

```
git switch testing
```

nano HW1/test_revert_merge.txt → ввожу желаемый текст ("This is a file for testing revert and merge. This is a change.")

```
git add HW1/test_revert_merge.txt
```

```
git commit -m "Update file for merge testing"
```

```
git push
```

13) Пытаемся снова слить ветки:

```
git switch HW1
```

```
git merge testing
```

Но ожидаемого результата, а именно слияния всех изменений, не произошло. Было зафиксировано только последнее изменение в test_reverte_merge.txt. Это произошло из-за того, что Git смотрит на историю коммитов и видит, что слияние происходило и изменение в test_reverte.txt было зафиксировано. Он почему-то не обращает внимание на последующую отмену этого изменения, ему важен сам факт выполненной когда-то операции, а она была.

14) Делаем revert revert'a и удаляем testing.

```
git log --oneline --graph --all --decorate
```

```
* 692cbb3 (HEAD -> HW1) Merge branch 'testing' into HW1
| \
| * adf235a (origin/testing, testing) Update file for merge testing
* | 801c801 (origin/HW1) Revert "Merge branch 'testing' into HW1"
* | 213529d Merge branch 'testing' into HW1
| \
| * 2384e2e Update file for revert testing
* | 91ef8e6 Update hw1 file
|/
* 1510ba6 Add initial files for HW1
* d274fe9 (origin/main, origin/HEAD, main) Initial commit
```

Тогда:

```
git revert 801c801
```

```
git branch -d testing
```

```
git push origin --delete testing
```

Часть 2. Основные команды

Команда	Объяснение
git --version	Показывает установленную версию Git.
git config --global user.name "Имя Фамилия"	Устанавливает имя автора коммитов.
git config --global user.email "email@example.com"	Устанавливает email автора коммитов.
git config --list	Показывает текущие настройки Git.
git clone <URL>	Клонирует удалённый репозиторий на компьютер.
git status	Показывает текущее состояние рабочей директории и индекса.
git add <file>	Добавляет изменения указанного файла в индекс, то есть подготавливает их к следующему коммиту.
git add .	Добавляет в индекс все текущие изменения в текущей директории и вложенных папках.
git add HW1/	Добавляет в индекс все изменения внутри папки HW1.
git diff	Показывает изменения, которые ещё не были добавлены в индекс.
git diff --staged	Показывает изменения, уже добавленные в индекс и готовые попасть в следующий коммит.
git commit -m "message"	Создаёт коммит из изменений, находящихся в индексе.
git branch	Показывает список локальных веток; текущая отмечается *.
git branch <branch>	Создаёт новую ветку, но не переключается на неё.
git switch <branch>	Переключается на указанную ветку.
git switch -c <branch>	Создаёт новую ветку и сразу переключается на неё.
git push	Отправляет локальные коммиты текущей ветки в связанный удалённый репозиторий.
git push -u origin <branch>	Впервые отправляет новую локальную ветку на origin и связывает её с удалённой веткой.
git merge <branch>	Объединяет указанную ветку с текущей веткой.
git revert <commit_hash>	Создаёт новый коммит, отменяющий изменения указанного коммита.
git revert -m 1 <merge_commit_hash>	Отменяет merge-коммит, считая первого родителя основной линией истории.
git log --oneline --graph --all --decorate	Показывает историю всех веток в виде графа с названиями веток и указателями.
git branch -d <branch>	Удаляет локальную ветку.